

Fourier Transforms and Denoising an Accelerometer Signal

Haotian Fang

TABLE OF CONTENTS

INTRODUCTION.....	3
INTRODUCTION TO THE FOURIER SERIES: MATHEMATICS AND METHODOLOGY.....	6
MATHEMATICS OF THE FOURIER SERIES.....	9
DERIVING THE FOURIER SERIES COEFFICIENTS.....	10
Deriving the a_0 coefficient.....	10
Deriving the a_n Coefficient.....	12
Deriving the b_n Coefficient.....	17
DISCRETE FOURIER TRANSFORM (DFT).....	21
FOURIER TRANSFORM ON AN ACCELEROMETER SIGNAL.....	23
Methodology.....	23
Data Set Context.....	24
Data Processing.....	25
Analyzing the Cutoffs for The Graphs.....	29
Observations.....	33
CONCLUSION.....	34
REFERENCES.....	36
APPENDIX A.....	37

INTRODUCTION

After participating in three years of robotics competitions, I have been observing an issue that has persisted throughout every season: autonomous coding discrepancies. For instance, when my team coder programs our robot to move a certain distance to reach a target location, it often stops short of the goal or overshoots. Furthermore, in order to stabilize the robot's motion and control its speed, we have found that the chassis often gets burnt out more than expected to regulate irregular driving motion. This has been a constant source of frustration for my team and we have spent large portions of time tuning this issue and adjusting the code to make it as accurate as possible. I realized that no matter which inertial sensor we used throughout the years, there always existed this discrepancy in what was coded and what was executed by the robot. Through further investigation, I came to learn about signal noise, as well as the concept of removing it—denoising.

My team's robot primarily relies on an inertial measurement unit (IMU), which contains a 3-axis (x,y,z) accelerometer and gyroscope. As the major issue at hand is robot stability and movement, this investigation will focus exclusively on data procured from an accelerometer. Accelerometers are often used in various real-world contexts, such as in smartphones and

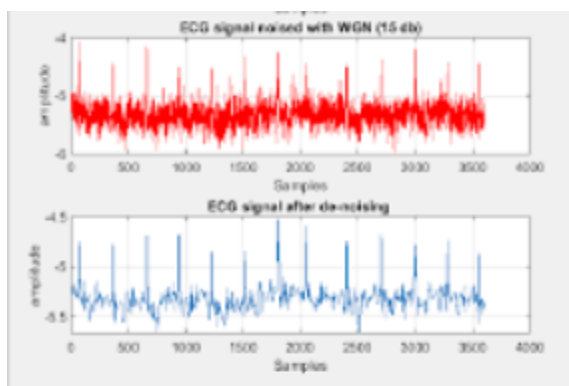


Figure 1

automobiles for a plethora of movement-related purposes. The accelerometer's signal contains "noise", unwanted components of the signal that are considered extraneous to the data being transferred through the signal. Data scientists,

engineers, or other professions dealing with signals are always seeking to "denoise" them for better output clarity, as the presence of noise in a signal can cause parts of the signal to corrupt and be unreadable. Denoising a signal is a term in signal processing in which the signal becomes cleaner through the removal of excess noise. This is especially important in the real world because the presence of noise in a signal can cause significant data transmission errors such as the aforementioned robotics autonomous discrepancies. Once such method employed for this purpose is signal denoising through Fourier analysis. Figure 1 shows the difference between noisy and denoised signals, with the denoised data plot having significantly less points plotted after the processing. My research question I have selected is, "How can the Fourier transforms be applied to signal denoising in an accelerometer?".

I will start the essay with an introduction of Fourier analysis as well as a breakdown of the math involved. Then, I will apply the math behind it to an unfiltered three-dimensional accelerometer dataset found from a machine learning/data science online database containing data collected every second for a total of 1752 seconds (approximately 30 minutes). The pre-processed graph of the original data points will then be compared to the denoised graph to show the difference in data clarity, ultimately demonstrating the extent of the effectiveness of Fourier analysis for signal denoising and exploring some other mathematical options for denoising.

Through the exploration of Fourier analysis and signal denoising in this EE, I hope to not only better understand the root causes of the inconsistencies in my robotics autonomous coding

but also about data science. I want to further grasp the mathematics applied in data science, and the particular effectiveness of the Fourier method I have chosen.

INTRODUCTION TO THE FOURIER SERIES: MATHEMATICS AND METHODOLOGY

Fourier analysis is the process of breaking down any periodic and continuous function into a series of sine and cosine functions. For an example to better demonstrate this, refer to the complex waveform in figure 2, $f(x) = 3\sin(4x) + 2\cos(3x)$, below:

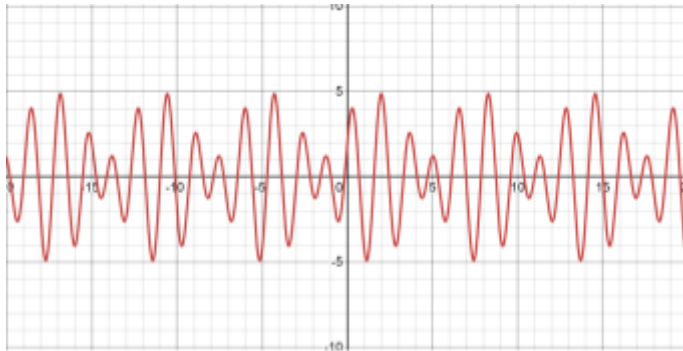


Figure 2, produced by author on Desmos

By applying Fourier analysis to the above waveform, a mathematical breakdown of the end result seen in Figure 2 could be deduced to find the function that produced it, $f(x)$. The graph is the spatial domain representation of $f(x)$, where a spatial domain is defined as the normal image space (Fisher et al., 2003), mapping a function with (x,y) coordinates to produce a 2D image on a 2D cartesian plane. In applying Fourier analysis to the spatial domain of $f(x)$, the spatial domain converts to a frequency domain, effectively breaking down a function into its respective frequency components and shows the amplitude of the sinusoidal functions in $f(x)$. To illustrate the information conveyed in the frequency domain of the $f(x)$ graphed in Figure 2, we must first derive information from the functions to get to the frequency domain:

Written in the form $f(x) = a\sin(bx)$ where:

- a is the amplitude of the term
- $\frac{2\pi}{|b|}$ is the period of the term

Similarly, with a cosine term written in the form $f(x) = a\cos(bx)$:

- a is the amplitude of the term
- $\frac{2\pi}{|b|}$ is the period of the term

Thus, for the equation $f(x) = 3\sin(4x) + 2\cos(3x)$:

For the term $3\sin(4x)$:

- Amplitude = 3
- Period = $\frac{2\pi}{4} = \frac{\pi}{2}$

For the term $2\cos(3x)$:

- Amplitude = 2
- Period = $\frac{2\pi}{3}$

To visualize these features, a bar graph can be drawn to represent the relationship between the period of the function and its amplitude in the frequency domain:

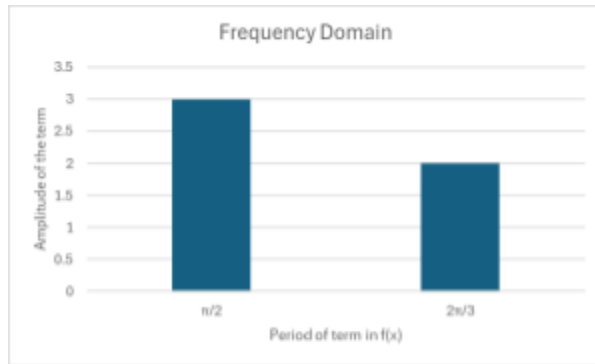


Figure 3, produced by author in Excel

The breakdown of a function into its sine and cosine components via Fourier Analysis allows the unnecessary high and low ends of the signal's frequency (noise) to be exposed. This cleaning of the noise from the frequency components is key to this investigation and overall Fourier analysis applications to denoising.

MATHEMATICS OF THE FOURIER SERIES

By the base definition of the Fourier series itself, the initial function for it, $f(x)$, is as shown:

$$F(x) = \sum_{n=0}^{\infty} \left[a_n \cos \frac{n\pi x}{L} + b_n \sin \frac{n\pi x}{L} \right]$$

Where the L is half the period of the trigonometric functions (the period is equal to $2L$).

To rearrange this in a different format, substituting $n = 0$ into the formula produces:

$$\begin{aligned} F(x) &= a_0 \cos \frac{0\pi x}{L} + b_0 \sin \frac{0\pi x}{L} \\ &= a_0 \cos(0) + b_0 \sin(0) \\ &= a_0 + 0 = a_0 \end{aligned}$$

As this substitution of $n = 0$ produces a single term in the function, the series can be rewritten:

$$F(x) = a_0 + \sum_{n=1}^{\infty} \left[a_n \cos \frac{n\pi x}{L} + b_n \sin \frac{n\pi x}{L} \right]$$

Equation 1

The next steps are to derive the coefficients present in the series, namely a_0 , a_n , and b_n .

DERIVING THE FOURIER SERIES COEFFICIENTS

Deriving the a_0 coefficient

General equations for each of the coefficients in the Fourier series can be derived from the overall formula. First, to derive the equation for the a_0 coefficient, both sides of the base Fourier series equation must be integrated. The range of integration should match the aforementioned overall period of the function, $2L$: we can thus integrate the base equation with respect to x from $-L$ to L . This range was selected because the area under a base (non-translated) sinusoidal function is 0 when the limit range of the integral is equal to the period of the function itself.

Integrating as such produces:

$$\int_{-L}^L F(x)dx = \int_{-L}^L a_0 dx + \sum_{n=1}^{\infty} \int_{-L}^L a_n \cos\left(\frac{n\pi x}{L}\right)dx + \sum_{n=1}^{\infty} \int_{-L}^L b_n \sin\left(\frac{n\pi x}{L}\right)dx$$

To rearrange the equation in terms of a_0 :

As mentioned above, the second and third terms on the right hand side (RHS) of the equation all simplify to 0:

$$\int_{-L}^L F(x)dx = \int_{-L}^L a_0 dx + 0 + 0$$

$$\int_{-L}^L F(x)dx = (a_0 x)_{-L}^L$$

$$\int_{-L}^L F(x)dx = a_0(L) - a_0(-L)$$

$$= 2a_0L$$

The equation in terms of a_0 is as such:

$$a_0 = \frac{1}{2L} \int_{-L}^L F(x)dx$$

Equation 2

Deriving the a_n Coefficient

The a_n coefficient can be derived from the first equation as well. As it is not isolated like a_0 was, we can use trigonometric identities to better simplify the equation and isolate for a_n .

This can be done by multiplying both sides of the base equation by $\cos(\frac{m\pi x}{L})$, where $m \in \mathbb{Z}$.

This gives:

$$\int_{-L}^L F(x) \cos(\frac{m\pi x}{L}) dx = \int_{-L}^L a_0 \cos(\frac{m\pi x}{L}) dx + \sum_{n=1}^{\infty} \int_{-L}^L a_n \cos(\frac{n\pi x}{L}) \cos(\frac{m\pi x}{L}) dx + \sum_{n=1}^{\infty} \int_{-L}^L b_n \sin(\frac{n\pi x}{L}) \cos(\frac{m\pi x}{L}) dx$$

The first term on the RHS of the equation produces a cosine wave with the standard period of $2L$, producing the same result of 0 as it is integrating on its period.

The second and third terms on the RHS can be simplified using the trigonometric identity of cosine summation and subtraction:

$$\cos(A + B) + \cos(A - B) = \cos A \cos B - \sin A \sin B + \cos A \cos B + \sin A \sin B$$

$$\cos(A + B) + \cos(A - B) = 2\cos A \cos B$$

$$\cos A \cos B = \frac{\cos(A+B) + \cos(A-B)}{2}$$

Referring back to the base equation, $A = \frac{n\pi x}{L}$, and $B = \frac{m\pi x}{L}$. This allows us to express the double cosine term (the second term on the RHS of the above equation) as a summation of cosine terms instead:

$$\cos(\frac{n\pi x}{L})\cos(\frac{m\pi x}{L}) = \frac{1}{2} [\cos(\frac{n\pi x}{L} + \frac{m\pi x}{L}) + \cos(\frac{n\pi x}{L} - \frac{m\pi x}{L})]$$

There are two cases to consider for this:

1. When $n \neq m$
2. When $n = m$

When $n \neq m$:

$$\begin{aligned} & \int_{-L}^L a_n \cos(\frac{n\pi x}{L}) \cos(\frac{m\pi x}{L}) dx \\ &= \frac{a_n}{2} \int_{-L}^L \cos(\frac{(m+n)\pi x}{L}) + \cos(\frac{(m-n)\pi x}{L}) dx \end{aligned}$$

Because $\int \cos(kx) dx = \frac{1}{k} \sin(kx)$ for any integer k ,

$$= \frac{a_n}{2} \left(\frac{L}{\pi(m+n)} \sin(\frac{(m+n)\pi x}{L}) + \frac{L}{\pi(m-n)} \sin(\frac{(m-n)\pi x}{L}) \right) \Big|_{-L}^L$$

As both m and n are integers, their sum and difference must also be integers. Thus:

$$\sin(\frac{(m+n)\pi x}{L}) = 0 \text{ when integrated over the above symmetrical limit}$$

$$\sin(\frac{(m-n)\pi x}{L}) = 0 \text{ when integrated over the above symmetrical limit}$$

For $k \in \mathbb{Z}$, the integral $\int_{-L}^L \sin(kx) dx = 0$ by symmetry. So both of the terms in the above equation

when integrated produces 0 as a result. The equation can thus be simplified:

$$= \frac{a_n}{2} \left(\frac{L}{\pi(m+n)} \cdot 0 + \frac{L}{\pi(m-n)} \cdot 0 \right)_{-L}^L$$

$$= 0$$

$$\int_{-L}^L a_n \cos\left(\frac{n\pi x}{L}\right) \cos\left(\frac{m\pi x}{L}\right) dx = 0 \text{ when } n \neq m.$$

When $n = m$:

$$\frac{a_n}{2} \int_{-L}^L \cos\left(\frac{(2m)\pi x}{L}\right) + \cos\left(\frac{(0)\pi x}{L}\right) dx$$

$$= \frac{a_n}{2} \int_{-L}^L \cos\left(\frac{(2m)\pi x}{L}\right) + 1 dx$$

$$= \frac{a_n}{2} \left(\frac{L}{2\pi m} \sin\left(\frac{(2m)\pi x}{L}\right) + x \right)_{-L}^L$$

$$= \frac{a_n}{2} (0 + 2L)$$

$$= \frac{a_n 2L}{2}$$

$$= a_n L$$

As there are two possible results to the initial integral, it can be summarized as such:

$$\int_{-L}^L a_n \cos\left(\frac{n\pi x}{L}\right) \cos\left(\frac{m\pi x}{L}\right) dx \begin{cases} 0, & \text{when } n \neq m \\ a_n L, & \text{when } n = m \end{cases}$$

The third term on the RHS of the overall equation can be simplified through the use of odd/even function knowledge and trigonometric identities.

$$\sum_{n=1}^{\infty} \int_{-L}^L b_n \sin\left(\frac{n\pi x}{L}\right) \cos\left(\frac{m\pi x}{L}\right) dx$$

As b_n is a constant when integrating with respect to x , it can be taken outside of the integral:

$$= \sum_{n=1}^{\infty} b_n \int_{-L}^L \sin\left(\frac{n\pi x}{L}\right) \cos\left(\frac{m\pi x}{L}\right) dx$$

Applying the trigonometric identity of the product identity:

$$\sin A \cos B = \frac{1}{2} (\sin(A + B) + \sin(A - B)) \text{ where in the integral } A = \frac{n\pi x}{L} \text{ and } B = \frac{m\pi x}{L}.$$

$$\sum_{n=1}^{\infty} b_n \int_{-L}^L \sin\left(\frac{n\pi x}{L}\right) \cos\left(\frac{m\pi x}{L}\right) dx \Rightarrow \sum_{n=1}^{\infty} b_n \int_{-L}^L \frac{1}{2} (\sin\left(\frac{n\pi x}{L} + \frac{m\pi x}{L}\right) + \sin\left(\frac{n\pi x}{L} - \frac{m\pi x}{L}\right)) dx$$

Here, the knowledge of odd and even functions can be applied to further simplify the term. An odd function is defined by its fulfillment of $f(-x) = -f(x)$, meaning that it remains unchanged when reflected across the origin. An even function is self-symmetric around the y-axis, meaning that it fulfills $f(-x) = f(x)$. The sine function is one such example of an odd function.

The integral above is integrating through two symmetric limits: it consists of the sum of two sine terms, namely two odd functions, which integrate to zero over such a symmetric limit. Thus the term simply becomes zero.

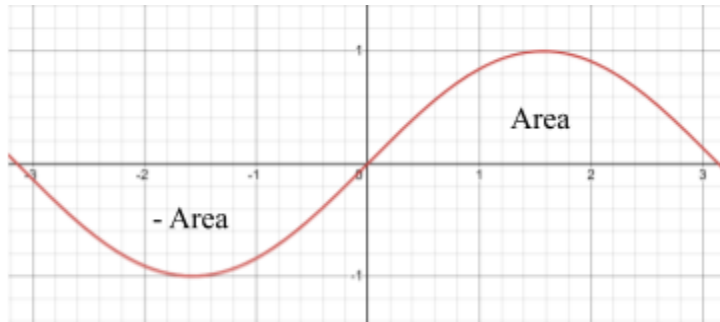


Figure 4: Illustrating how integrating an odd function over symmetric limits simplifies to zero

To reinsert all of these simplified terms back into the original equation:

$$\begin{aligned}
 \int_{-L}^L F(x) \cos\left(\frac{m\pi x}{L}\right) dx &= \int_{-L}^L a_0 \cos\left(\frac{m\pi x}{L}\right) dx + \sum_{n=1}^{\infty} \int_{-L}^L a_n \cos\left(\frac{n\pi x}{L}\right) \cos\left(\frac{m\pi x}{L}\right) dx + \sum_{n=1}^{\infty} \int_{-L}^L b_n \sin\left(\frac{n\pi x}{L}\right) \cos\left(\frac{m\pi x}{L}\right) dx \\
 &= 0 + a_n L + 0 \\
 &= a_n L
 \end{aligned}$$

Rearranging for a_n :

$$a_n = \frac{\int_{-L}^L F(x) \cos\left(\frac{m\pi x}{L}\right) dx}{L}$$

$$a_n = \frac{1}{L} \int_{-L}^L F(x) \cos\left(\frac{m\pi x}{L}\right) dx$$

Equation 3

Deriving the b_n Coefficient

The b_n coefficient can be similarly derived like the a_n coefficient, but multiplying both sides of the equation by $\sin(\frac{m\pi x}{L})$, where $m \in \mathbb{Z}$ and integrating with respect to x . This gives:

$$\int_{-L}^L F(x) \sin(\frac{m\pi x}{L}) dx = \int_{-L}^L a_0 \sin(\frac{m\pi x}{L}) dx + \sum_{n=1}^{\infty} \int_{-L}^L a_n \sin(\frac{n\pi x}{L}) \cos(\frac{m\pi x}{L}) dx + \sum_{n=1}^{\infty} \int_{-L}^L b_n \sin(\frac{n\pi x}{L}) \sin(\frac{m\pi x}{L}) dx$$

Notice that in the first term of the RHS of the equation, it contains $\sin(\frac{m\pi x}{L})$, which integrates to zero as it is integrating over the range of its period, $2L$.

The second term on the RHS is a product of a sine and cosine, which is an odd function multiplied by an even function. An odd function multiplied by an even function will always produce an odd function, which integrates to zero when integrating symmetrical limits.

Thus, the overall equation simplifies to:

$$\int_{-L}^L F(x) \sin(\frac{m\pi x}{L}) dx = 0 + 0 + \sum_{n=1}^{\infty} \int_{-L}^L b_n \sin(\frac{n\pi x}{L}) \sin(\frac{m\pi x}{L}) dx$$

To simplify the remaining term on the RHS, a technique similar to that of the a_n coefficient derivation can be employed to simplify the multiplication of two sine terms. Here, the trigonometric identity to be used is the compound angle identity.

$$\cos(A - B) - \cos(A + B) = \cos A \cos B + \sin A \sin B - \cos A \cos B + \sin A \sin B = 2 \sin A \sin B$$

$$\sin A \sin B = \frac{1}{2} (\cos(A - B) - \cos(A + B))$$

In the context of the equation, $A = \frac{n\pi x}{L}$, and $B = \frac{m\pi x}{L}$:

$$\sin \frac{n\pi x}{L} \sin \frac{m\pi x}{L} = \frac{1}{2} (\cos(\frac{n\pi x}{L} - \frac{m\pi x}{L}) - \cos(\frac{n\pi x}{L} + \frac{m\pi x}{L}))$$

Similarly to the a_n coefficient derivation, two cases must be considered: $n \neq m$ and $n = m$.

Considering the $n \neq m$ case first with reverse chain rule:

$$\begin{aligned} & \int_{-L}^L b_n \sin\left(\frac{n\pi x}{L}\right) \sin\left(\frac{m\pi x}{L}\right) dx \\ &= \frac{1}{2} b_n \int_{-L}^L \cos\left(\frac{n\pi x}{L} - \frac{m\pi x}{L}\right) - \cos\left(\frac{n\pi x}{L} + \frac{m\pi x}{L}\right) dx \\ &= \frac{1}{2} b_n \left(\frac{L}{\pi(n-m)} \sin\left(\frac{n\pi x}{L} - \frac{m\pi x}{L}\right) - \frac{L}{\pi(n+m)} \sin\left(\frac{n\pi x}{L} + \frac{m\pi x}{L}\right) \right)_{-L}^L \end{aligned}$$

Recall that as both m and n are integers, their sum and difference must also be integers.

$$\sin\left(\frac{(m+n)\pi x}{L}\right) = 0 \text{ when integrated over the above symmetrical limit}$$

$$\sin\left(\frac{(m-n)\pi x}{L}\right) = 0 \text{ when integrated over the above symmetrical limit}$$

Since $\sin(kx) = 0$ where k is any given integer. So both of the terms in the above equation when integrated produces 0 as a result:

$$\int_{-L}^L b_n \sin\left(\frac{n\pi x}{L}\right) \sin\left(\frac{m\pi x}{L}\right) dx = 0$$

Now considering the second case, $n = m$:

$$\begin{aligned} & \frac{b_n}{2} \int_{-L}^L \cos\left(\frac{(0)\pi x}{L}\right) + \cos\left(\frac{(0)\pi x}{L}\right) dx \\ &= \frac{b_n}{2} \int_{-L}^L 1 - \cos\left(\frac{(2m)\pi x}{L}\right) dx \\ &= \frac{b_n}{2} \left(x + \frac{L}{2\pi m} \sin\left(\frac{(2m)\pi x}{L}\right) \right)_{-L}^L \\ &= \frac{b_n 2L}{2} \\ &= b_n L \end{aligned}$$

As there are again two possible results to the initial integral, it can be summarized as such:

$$\int_{-L}^L b_n \sin\left(\frac{n\pi x}{L}\right) \sin\left(\frac{m\pi x}{L}\right) dx \begin{cases} 0, \text{ when } n \neq m \\ b_n L, \text{ when } n = m \end{cases}$$

Going back to the overall equation:

$$\begin{aligned} \int_{-L}^L F(x) \sin\left(\frac{m\pi x}{L}\right) dx &= \int_{-L}^L a_0 \sin\left(\frac{m\pi x}{L}\right) dx + \sum_{n=1}^{\infty} \int_{-L}^L a_n \sin\left(\frac{n\pi x}{L}\right) \cos\left(\frac{m\pi x}{L}\right) dx + \sum_{n=1}^{\infty} \int_{-L}^L b_n \sin\left(\frac{n\pi x}{L}\right) \sin\left(\frac{m\pi x}{L}\right) dx \\ &= 0 + 0 + b_n L \\ &= b_n L \end{aligned}$$

Rearranging for the b_n coefficient:

$$b_n = \frac{1}{L} \int_{-L}^L F(x) \sin\left(\frac{n\pi x}{L}\right) dx$$

Equation 4

DISCRETE FOURIER TRANSFORM (DFT)

As the data being investigated in the real world is from data output sources such as sensor signals, the data is being collected on a sample-by-sample basis, meaning that it produces a data point per unit time. This means that the data is classified as discrete data--- data that is separated point by point, rather than continuous. However, the Fourier Series works with continuous functions rather than discrete, and as such, a transform called the Discrete Fourier Transform

(DFT) is used instead in practice to apply a similar analysis tailored towards the discrete functions in the real world.

$$X_k = \sum_{n=0}^{N-1} x_n e^{-2\pi i \frac{kn}{N}}, \text{ where } n = 0, 1, \dots, N - 1. \text{ (Smith, 2007)}$$

Where:

- n is the sample number (time index)
- x_n is the n th sampled data point
- N is the total number of samples being collected
- k is the frequency index
- X_k is the Fourier coefficient at a value of k
- $e^{-2\pi i \frac{kn}{N}}$ is the complex exponential, which can be explained by Euler's formula

Euler's formula:

$$e^{i\theta} = \cos\theta + i\sin\theta$$

The exponential term in the above equation thus represents the summation of a cosine and sine term, at the index frequency value of k .

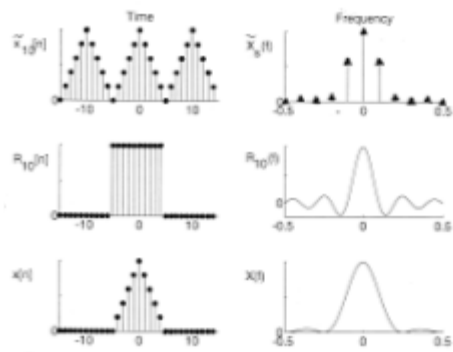


Figure 5: Illustrating the concept of a DFT, (Delgutte & Greenberg, 2005)

FOURIER TRANSFORM ON AN ACCELEROMETER SIGNAL

Methodology

As the data presented to data scientists and engineers often come in extremely large numbers, the Fourier series is not usually performed manually because of this. Instead, coding programs are written to perform these operations in large amounts and in short periods of time. The coding algorithms use a method called Fast Fourier Transforms (FFT), where the algorithm is able to perform the DFT onto the data sets and thereby break down data into its frequency components.

I will first use a code written in Python to apply the DFT using the FFT algorithm onto the three-axis accelerometer dataset, which will thereby produce the frequency values per sample for each of the axes. After this, I will look at all of the frequency graphs produced and determine where the noise is on the graph--- this can be done by looking for irregular spikes in the graph or a drastic flattening of the data, mostly at the beginning and ends of the data.

After the high and low frequency cutoffs for noise have been determined, I will then re-apply the Fourier transform with code onto the original dataset with the noise components removed, thereby producing a more filtered signal processed via the Fourier transform.

This re-application of the Fourier transform is known as the Inverse Discrete Fourier Transform (IDFT), where the process is applied backwards from a frequency breakdown to the context of the original dataset. A sort of "bandpass filtering" is applied in the process so that the IDFT is performed after the signal has been cleaned, allowing only the desired "bands" of frequency to pass. This reversal to obtain cleaned data is called Fourier synthesis.

Data Set Context

Although my direct purpose for this investigation was set in the context of my robotics team and robot sensors, the sensors used in the competition program I am in are unable to transmit data onto a device such as a computer. As such, I decided that finding a sensor database for the same type of sensor, accelerometer, would suffice for the overall meaning of this investigation.

My standards for finding the appropriate data set is as follows:

- Should be three-axis data to better model the type of accelerometers used in robotics
- Must be unfiltered/unprocessed data to allow for me to perform cleaning processes onto it
- Should have sufficient data (>100 samples) to have more data to perform Fourier analysis on

As such, I followed these requirements as I looked through databases for the appropriate dataset. I eventually found an appropriate one on Kaggle, the largest data science community platform online, with 3-axis accelerometer data implemented on an iPhone 6s. It contains 1752 sample points with a sample frequency of one sample per second, which is sufficient for the scope of this investigation.

Data Processing

To complete this investigation, I decided that I wanted to see the before and after graphical results of the data.

Graphing the raw data:

As these data points naturally overlap between the axes, the graphical representations of each separate axes data is graphed:

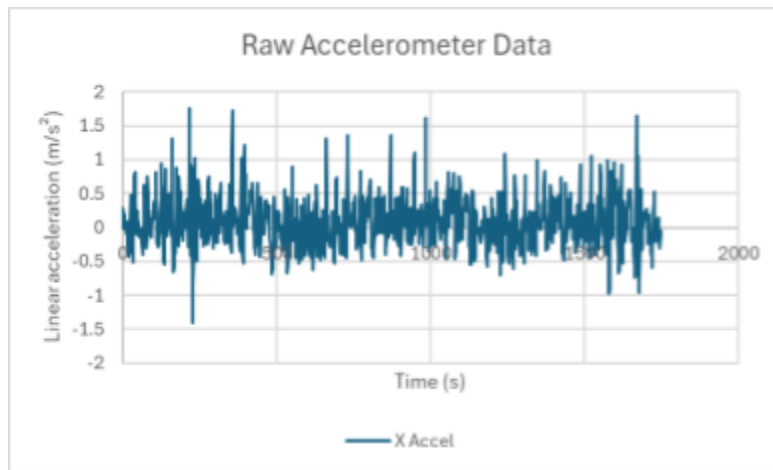


Figure 6: Raw x-axis accelerometer data graphed, created by author on Excel

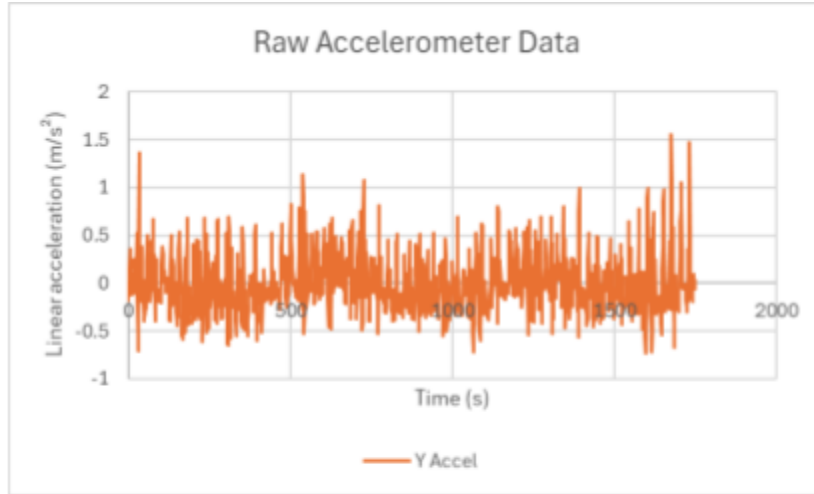


Figure 7: Raw y-axis accelerometer data, graphed by author on Excel

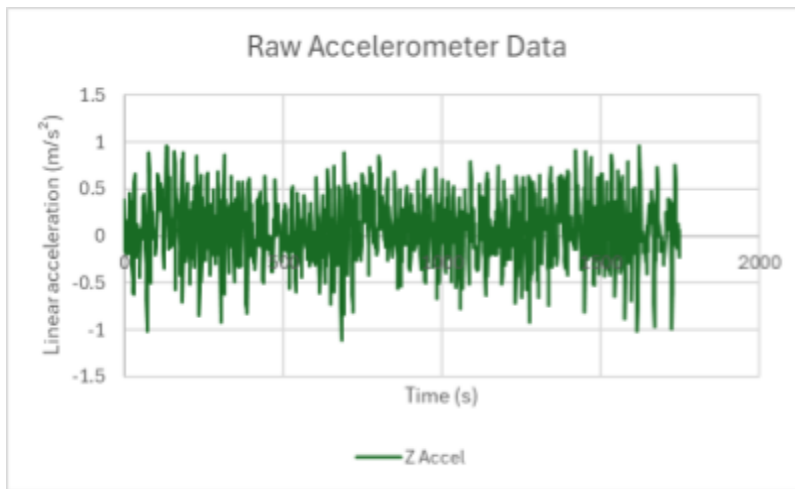


Figure 8: Raw z-axis accelerometer data, created by author on Excel

Then, to perform Fourier analysis onto each axis component of the accelerometer signal, the `fft` function in the NumPy library in Python can be applied to the dataset.

The graphs produced by the code created by generative AI are as such (see Appendix A for the code used):

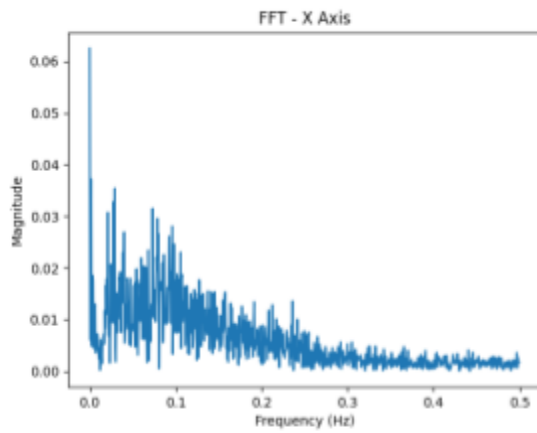


Figure 9: Fourier analysis performed on the x-axis accelerometer data using Python code created by generative AI.

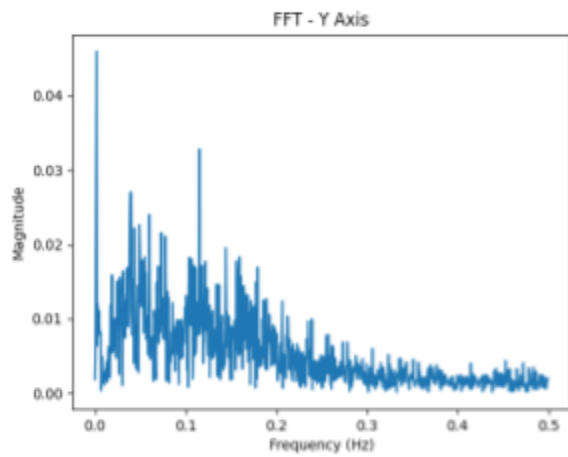


Figure 10: Fourier analysis performed on the y-axis accelerometer data using Python code created by generative AI.

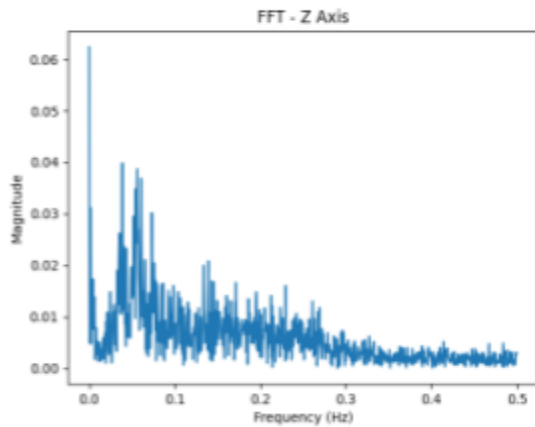
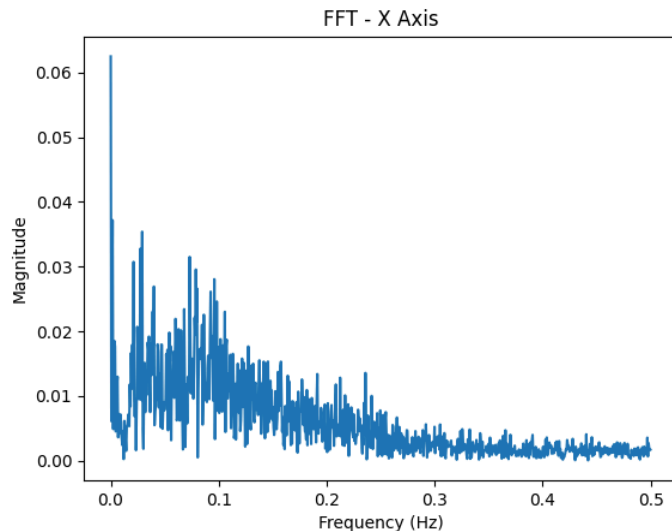


Figure 11: Fourier analysis performed on the z-axis accelerometer data using Python code created by generative AI.

These graphs were produced using the `fft` function in the Python NumPy library, and factoring in sampling frequency.

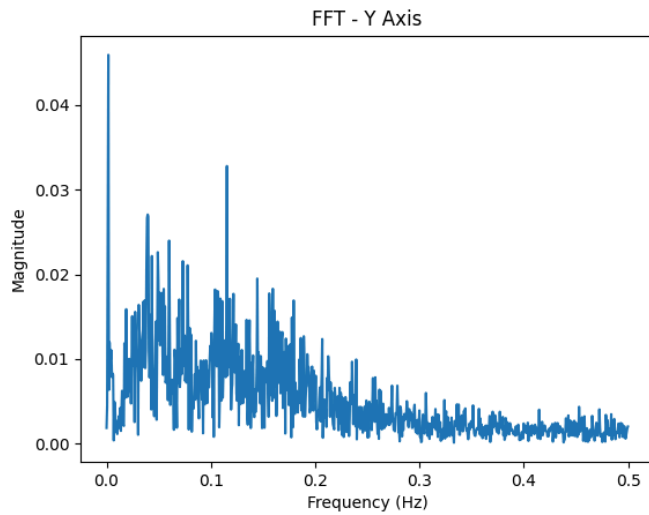
Sampling frequency is the number of samples that are collected per increment in time. This is defined in the code by $fs = 1 / dt$, where fs is the sampling frequency and dt is the difference in time between adjacent samples, defined as $time[1] - time[0]$. As the dataset collected one sample per second, the sampling frequency was equal to 1 Hz.

Analyzing the Cutoffs for The Graphs



In the above x-axis Fourier analysis graph, we can now determine the cutoffs for useful signal and noise. To do this, we can look at sudden spikes in the frequency graph (typically at the beginning), and then an overly flat trendline towards the ends of the graphs. This indicates abnormally high or low and stagnant frequencies, which can be discarded as the noise in the signal.

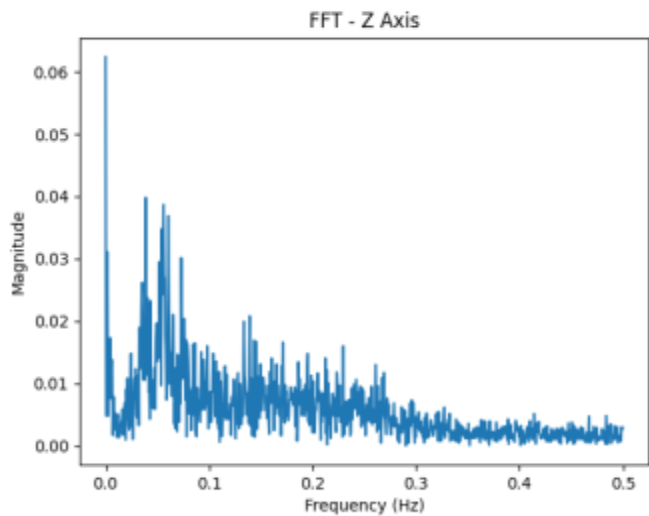
In the above graph: There is an abnormal spike at around the 0.0 Hz point, following with some continuing spikes that end at approximately the 0.1 Hz point. Thus, for this graph, an appropriate low cutoff would be the 0.1 Hz mark. For the high cutoff, the frequency decreases consistently in magnitude until a little before the 0.3 Hz mark, where it becomes stagnant past that point. The appropriate high cutoff for this graph would be the 0.3 Hz point.



Using the methodology for the above cutoff determination:

Low cutoff: slightly after 0.1Hz, where there is a significant spike in frequency magnitude

High cutoff: after 0.3Hz, where the frequency becomes static and low in magnitude.



Low cutoff: before the 0.1Hz point, where the last major spike in frequency is located.

High cutoff: After the 0.3 Hz mark, where the frequency magnitude hovers at a very low magnitude.

Because the code being used only has one overall cutoff variable, the approximated averages for low and high cutoffs were taken.

Overall low cutoff: 0.1 Hz

Overall high cutoff: 0.3 Hz

Inputting this into the code, the reverse Fourier transform (rfft function) was performed. The following are the denoised graphs in orange, overlaid on the original raw data graphs in blue.

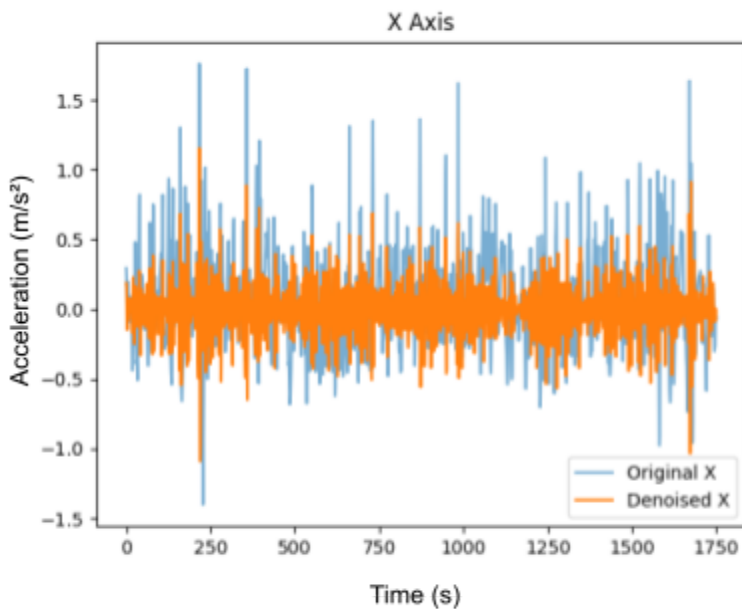


Figure 12: Denoised x-axis graph and raw data graph, produced by generative AI in Python

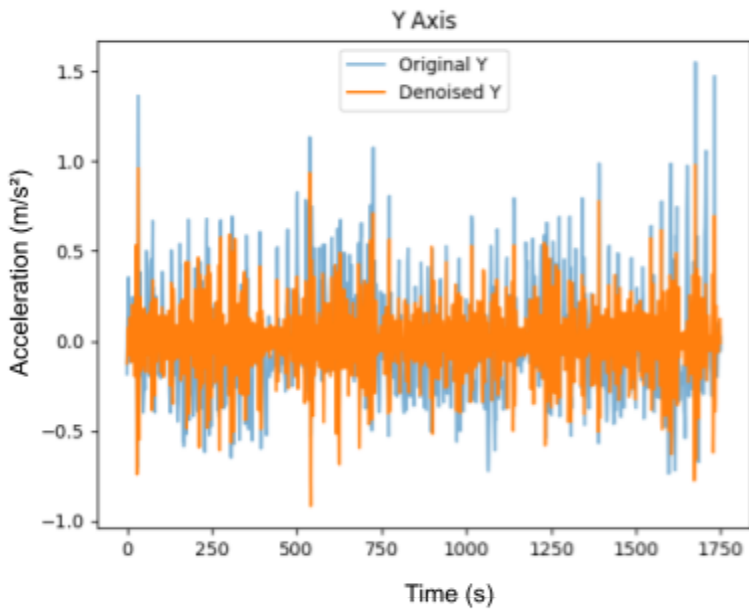


Figure 13: Denoised y-axis graph and raw data graph, produced by generative AI in Python

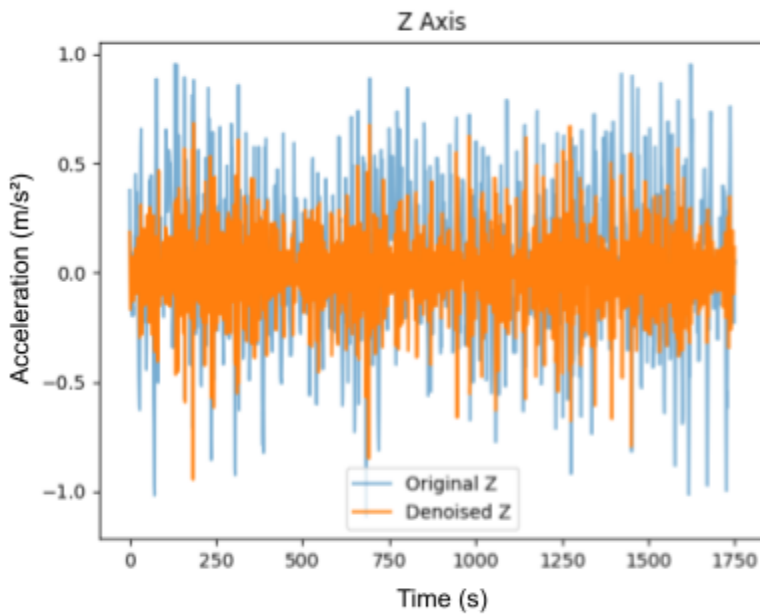


Figure 14: Denoised z-axis graph and raw data graph, produced by generative AI in Python

Observations

In each of these graphs, there has been a significant reduction in the oscillations of high-frequency signal data, and successfully smoothing out the magnitude of the spikes in data. The overall data trend is still preserved, which maintains the signal's original integrity.

The original range of the accelerations largely decreased:

X-axis original range: -1.4 m/s^2 to 2.0 m/s^2

Y-axis original range: -0.9 m/s^2 to 1.4 m/s^2

Z-axis original range: -0.9 m/s^2 to almost 1.0 m/s^2

X-axis denoised range: -1.2 m/s^2 to 1.2 m/s^2

Y-axis denoised range: -0.9 m/s^2 to 0.9 m/s^2

Z-axis denoised range: -0.9 m/s^2 to 0.7 m/s^2

CONCLUSION

The overall result of the experiment section of the investigation was that the signal's large fluctuations were effectively reduced and the overall trend of the graph is still maintained. However, much of this is compromised in the overall effectiveness of the investigation because the raw and denoised data was unable to be implemented onto an actual accelerometer and tested for the empirical differences between the two. This would be the most direct and real-world assessing of this method and its overall success.

Additionally, an important potential tradeoff to note is that some important signal components may have been cut off as well. When deciding the high and low frequency cutoffs, I merely did it based off of approximate large inconsistencies in the graph, and the stagnancy in the frequency towards the ends. Additionally, the cutoffs were determined for all three axis graphs as a whole, rather than being tailored towards each individual graph which would have made the cutoffs more accurate overall. This means that in some areas, too much frequency may have been cut off, resulting in a loss of actual useful signal data, which can cause more inaccuracies in signal transmission.

To improve the effectiveness and purpose of the investigation, implementation of the datasets onto an actual sensor would largely increase the overall measure of the successes and limitations to the method. A dataset with a higher sampling frequency would also be more beneficial to the overall accuracy of the investigation, as according to the Nyquist frequency (Hasegawa-Johnson, 2023), the highest accuracy of frequency is directly dependent on sampling frequency. A sampling frequency of 1 Hz, as seen in this investigation, is considered relatively low. Additionally, it would also be appropriate to investigate more into other forms of signal

denoising that are applied by data scientists and engineers in the real world, and compare it to that of the Fourier method. This way, the advantages and disadvantages to each method could be analyzed for a more holistic view on denoising data overall and serve to be more effective in denoising specific datasets.

REFERENCES

Delgutte, B., & Greenberg, J. (2005). HST582J/6.555J/16.456J Biomedical Signal and image processing spring 2005. https://web.mit.edu/~gari/teaching/6.555/lectures/ch_DFT.pdf

Department of Mathematics | Rice University . (n.d.-a). CHAPTER 12 Fourier Series.

<https://math.rice.edu/~polking/math322/chapter12a.pdf>

Fisher, R., Perkins, S., Walker, A., & Wolfart, E. (2003). *Spatial domain*. Glossary - Spatial Domain. <https://homepages.inf.ed.ac.uk/rbf/HIPR2/spatdom.htm>

Hasegawa-Johnson, M. (2023). Review sampling aliasing aliased frequency aliased phase summary example. <https://courses.grainger.illinois.edu/ece401/fa2023/slides/lec06.pdf>

MIT. (n.d.). Chapter 4 Fourier series and Integrals 4.1.

<https://math.mit.edu/~gs/cse/websections/cse41.pdf>

Smith, J. O. (2007). *The Discrete Fourier Transform (DFT)*. CCRMA.

https://ccrma.stanford.edu/~jos/st/Discrete_Fourier_Transform_DFT.html

University of Auckland . (n.d.). *Spatial Frequency Domain*. Spatial frequency domain.

<https://www.cs.auckland.ac.nz/courses/compsci773s1c/lectures/ImageProcessing-html/topic1.htm>

University of California, Berkeley . (n.d.-b). 10.3 fourier series.

https://math.berkeley.edu/~arash/54/notes/10_3.pdf

APPENDIX A

Code used to perform Fourier transform and filtered results:

```
1  import numpy as np
2  import pandas as pd
3  import matplotlib.pyplot as plt
4
5  # === LOAD EXCEL FILE ===
6  df = pd.read_excel("C:\\Users\\[REDACTED]\\Downloads\\accelerometer_data.xlsx")
7
8  # Extract data
9  time = df["time"].values
10 x = df["x"].values
11 y = df["y"].values
12 z = df["z"].values
13
14 # === SAMPLING INFO ===
15 dt = time[1] - time[0]      # sampling interval
16 fs = 1 / dt
17 N = len(x)
18
19 # === FFT FUNCTION (Real-valued signal) ===
20 def compute_fft(signal):
21     fft_vals = np.fft.rfft(signal)
22     fft_vals = np.abs(fft_vals) / N
```

```

23     freqs = np.fft.rfftfreq(N, dt)
24     return freqs, fft_vals
25
26     # Compute FFT for each axis
27     freqs_x, fft_x = compute_fft(x)
28     freqs_y, fft_y = compute_fft(y)
29     freqs_z, fft_z = compute_fft(z)
30
31     # === PLOT ===
32     plt.figure()
33     plt.plot(*args: freqs_x, fft_x)
34     plt.title("FFT - X Axis")
35     plt.xlabel("Frequency (Hz)")
36     plt.ylabel("Magnitude")
37     plt.show()
38
39     plt.figure()
40     plt.plot(*args: freqs_y, fft_y)
41     plt.title("FFT - Y Axis")
42     plt.xlabel("Frequency (Hz)")
43     plt.ylabel("Magnitude")

```

```

43     plt.ylabel("Magnitude")
44     plt.show()
45
46     plt.figure()
47     plt.plot(*args: freqs_z, fft_z)
48     plt.title("FFT - Z Axis")
49     plt.xlabel("Frequency (Hz)")
50     plt.ylabel("Magnitude")
51     plt.show()
52
53
54     x = df["x"].values
55     y = df["y"].values
56     z = df["z"].values
57
58     # === Sampling frequency ===
59     fs = 1
60     dt = 1 / fs

```

```

61
62
63 # === Band-pass filter function ===
64 def bandpass_filter(signal, fs, low_cutoff, high_cutoff):
65     N = len(signal)
66     dt = 1 / fs
67
68     # Remove DC offset
69     signal = signal - np.mean(signal)
70
71     # FFT
72     fft_vals = np.fft.rfft(signal)
73     freqs = np.fft.rfftfreq(N, dt)
74
75     # Apply band-pass filter
76     fft_vals[(freqs < low_cutoff) | (freqs > high_cutoff)] = 0
77
78     # Inverse FFT (reconstruct signal)
79     filtered_signal = np.fft.irfft(fft_vals)
80
81     return filtered_signal

```

```

82
83
84 # === Choose cutoffs ===
85 low_cutoff = 0.1 # remove very low drift (Hz)
86 high_cutoff = 0.3 # remove high-frequency noise (Hz)
87
88 # === Apply to all axes ===
89 x_denoised = bandpass_filter(x, fs, low_cutoff, high_cutoff)
90 y_denoised = bandpass_filter(y, fs, low_cutoff, high_cutoff)
91 z_denoised = bandpass_filter(z, fs, low_cutoff, high_cutoff)
92
93 # === Plot comparison for each axis ===
94 plt.figure()
95 plt.plot(*args: x, label="Original X", alpha=0.6)
96 plt.plot(*args: x_denoised, label="Denoised X")
97 plt.legend()
98 plt.title("X Axis")
99 plt.show()
100
101 plt.figure()
102 plt.plot(*args: y, label="Original Y", alpha=0.6)
103 plt.plot(*args: y_denoised, label="Denoised Y")

```

```
104 plt.legend()
105 plt.title("Y Axis")
106 plt.show()
107
108 plt.figure()
109 plt.plot(*args: z, label="Original Z", alpha=0.6)
110 plt.plot(*args: z_denoised, label="Denoised Z")
111 plt.legend()
112 plt.title("Z Axis")
113 plt.show()
```